

Reviewer Pack — Minimal Rank of Tropicalized Lie Group Actions Bounds Quantu...

Entry 0d739d960283 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 06:02:52 UTC

Minimal Rank of Tropicalized Lie Group Actions Bounds Quantum Circuit Size

Entry ID: 0d739d960283 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 06:02:52 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Lie Groups) **Field B** (complexity object): Complexity Theory (Quantum Circuit Complexity)

Statement:

[‘Let G be a finite, non-abelian Lie group acting on an n -dimensional vector space V . Consider the tropicalization of the invariant theory associated with this action. Then the minimal rank of the tropicalized invariant vectors is upper-bounded by the size of a quantum circuit that can simulate the action of G on V up to polynomially bounded error.’, ‘Equivalently, for any Lie group action G on an n -dimensional vector space V , there exists a quantum circuit C with at most $R(G, V)$ gates such that the output of C is invariant under the action of G , where $R(G, V)$ is the minimal rank of the tropicalized invariant vectors.’, ‘For all instances with $n \leq 40$ and any finite, non-abelian Lie group action G on V , if the quantum circuit size of G is greater than $R(G, V)$, then there exists a counterexample to this conjecture.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry has been successfully used to study properties of algebraic varieties. By extending this to Lie groups, we can analyze the structure of group actions on vector spaces. This might reveal new connections between tropical geometry and quantum circuit complexity, potentially leading to new insights into the design and analysis of quantum algorithms.’, ‘The action of a Lie group on a vector space can be represented as unitary transformations in quantum mechanics. Therefore, understanding the minimal rank of the tropicalized invariant vectors could provide a novel approach for bounding the size of quantum circuits that simulate such actions.’]

Taxonomy category: TROPICAL_GEOMETRY_LIE_GROUPS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5112283417191232

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of generated Lie group actions on n -dimensional vector spaces ($n \leq 40$) have a quantum circuit size less than or equal to the minimal rank of tropicalized invariant vectors, and no instance with a known quantum circuit size larger than the predicted minimal rank is found. The conjecture is falsified if any instance has a quantum circuit size greater than the predicted minimal rank.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Lie group actions" AND "quantum circuit complexity" - "minimal rank" AND "tropicalization" AND "quantum circuit size" - "invariant vectors" AND "tropical geometry" AND "quantum circuit simulation"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_lie_group(n):
        # Placeholder for Lie group generation
        # This is a dummy implementation and should be replaced with actual logic
        if n == 2:
            return [[1, 0], [0, 1]]
        elif n == 3:
            return [[math.cos(math.pi/4), -math.sin(math.pi/4)], [math.sin(math.pi/4), math.cos(math.pi/4)]]
        else:
            raise NotImplementedError("Mapping_undefined")

    def matrix_mult(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        result = [[sum(A[i][k] * B[k][j] for k in range(p)) for j in range(n)] for i in range(m)]
```

```

    return result

def matrix_power(M, exp):
    if exp == 0:
        return [[Fraction(1) if i == j else Fraction(0) for j in range(len(M))] for i in range(len(M))]
    elif exp == 1:
        return M
    elif exp % 2 == 0:
        half_power = matrix_power(M, exp // 2)
        return matrix_mult(half_power, half_power)
    else:
        return matrix_mult(M, matrix_power(M, exp - 1))

def generate_vector(n):
    return [random.randint(-10, 10) for _ in range(n)]

def tropicalize(vector):
    return max(vector)

n = random.choice([5, 10, 15, 20, 30, 40])
G = generate_lie_group(n)
v = generate_vector(n)

invariant_vectors = [tropicalize(matrix_mult(G, v)) for _ in range(10)]
min_rank = min(invariant_vectors)

quantum_circuit_size = len(G) * n

return {
    "metric_name": "Quantum Circuit Size",
    "metric_value": quantum_circuit_size,
    "instances_tested": 10,
    "conjecture_holds": quantum_circuit_size <= min_rank,
    "counterexample": "" if quantum_circuit_size <= min_rank else f"Quantum circuit size {quantum_circuit_size} > minimal rank {min_rank}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Quantum circuit size > minimal rank\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9333867.py", line 77, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9333867.py", line 56, in run_trial
    G = generate_lie_group(n)
       ^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9333867.py", line 29, in generate_lie
    raise NotImplementedError("Mapping_undefined")
```

NotImplementedError: Mapping_undefined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result. | next: Investigate the cause of the crash and ensure that the test can run to completion. If the pre-registered support condition is met after addressing the issue, re-run the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22615
2	preregistration	ollama_remote	glm4:latest	0	0	5957
3	novelty	ollama_remote	glm4:latest	0	0	4813
4	novelty	ollama_remote	glm4:latest	0	0	5275
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12106
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11929
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12208
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9915
9	judge	ollama_remote	glm4:latest	0	0	10584

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 95401 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0d739d960283.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0d739d960283.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0d739d960283.tar.gz` (if generated)